Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Pemanfaatan Strategi *Divide and Conquer* untuk *Voxelization* Objek 3D menggunakan Octree

Semester II Tahun 2025/2026

foto-kelompok.png

Muhammad Fatih Irkham Mauludi (13524004)

Azri Arzaq Pohan (13524139)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Tugas **3**

2 Landasan Teori **5**

2.1 Divide and Conquer 5

2.2 Teorema Sumbu Pemisah 7

2.3 Octree dan Voxelization 8

2.4 Parallel Computing 9

2.5 3D Visualization 10

3 Impelementasi **14**

3.1 Voxelization 14

3.2 Concurrency 15

4 Hasil Eksperimen **17**

1 Deskripsi Tugas

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

Dalam tugas ini, perlu dilakukan implementasi sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari voxel dengan memanfaatkan properti Octree. Program melakukan konversi model dengan menggunakan algoritma *divide and conquer*.

Spesifikasi tugas perlu memperhatikan ketentuan-ketentuan berikut,

- Implementasi program perlu dibuat dalam bahasa pemrograman Go, C, C++, atau Java yang mengimplementasikan algoritma Divide and Conquer untuk melakukan konversi model tiga dimensi.
- Output voxel harus berukuran uniform, diambil dari leaf pada depth maksimum
- Transformasi hanya perlu dilakukan pada permukaan objek
- Tugas dikerjakan berkelompok dengan anggota 2 orang, boleh lintas kelas maupun kampus.
- Input: Program menerima path dari sebuah file .obj yang ingin dikonversi dan juga tingkat kedalaman pohon maksimum. Untuk tugas besar ini, hanya memperhatikan nilai x, y, z pada vertice v, dan faces f. Program harus bisa menentukan apakah input valid atau tidak. Format .obj valid adalah row-row yang berisikan <v x y z> atau <f i j k>.
- Output: Program mengembalikan: sebuah file .obj yang merupakan hasil konversi
- Di CLI, program harus menampilkan setidaknya informasi-informasi berikut:
  - Banyaknya voxel yang terbentuk
  - Banyaknya vertex yang terbentuk
  - Banyaknya faces yang terbentuk

  - Statistik node octree yang terbentuk
  - Statistik node yang tidak perlu ditelusuri
  - Kedalaman octree
  - Lama waktu program berjalan
  - Path dimana file .obj disimpan

- Spesifikasi bonus yang dapat diimplementasikan yaitu,
  - Menggunakan concurrency dalam bahasa pemrograman yang dipilih untuk algoritma konversi
  - Mengimplementasikan .obj viewer minimal menampilkan model 3D hasil voxelization secara interaktif dalam bahasa pemrograman yang dipilih untuk menampilkan hasil objek yang dikonversi

2 Landasan Teori

2.1 Divide and Conquer

Divide and Conquer adalah salah satu strategi algoritma dalam pemecahan permasalahan. Sesuai namanya, paradigma ini menggunakan beberapa tahapan-tahapan dalam mencari solusi. Tahapan tersebut diantaranya [**?**],

- Divide: membagi persoalan menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan semula namun berukuran lebih kecil (idealnya setiap upa-persoalan berukuran hampir sama),
- Conquer: menyelesaikan (solve) masing-masing upa-persoalan (secara langsung jika sudah berukuran kecil atau secara rekursif jika masih berukuran besar).
- Combine: menggabungkan solusi masing-masing upa-persoalan sehingga membentuk solusi persoalan semula.

Obyek persoalan yang dibagi dalam bentuk masukan (input) atau instances persoalan yang berukuran n seperti:

- tabel (larik),
- matriks,
- eksponen,
- polinom,

Tiap-tiap upa-persoalan memiliki karakteristik yang sama (*the same type*) dengan karakteristik persoalan semula namun berukuran lebih kecil sehingga metode Divide and Conquer lebih natural diungkapkan dalam skema rekursif.

Beberapa persoalan klasik yang dapat diselesaikan dengan strategi Divide and Concure yaitu,

1. Persoalan MinMaks (mencari nilai minimum dan nilai maksimum)
2. Menghitung perpangkatan
3. Persoalan pengurutan (sorting) – Mergesort dan Quicksort
4. Mencari sepasang titik terdekat (closest pair problem)
5. Convex Hull

6. Perkalian matriks
7. Perkalian bilangan bulat besar
8. Perkalian dua buah polinom
9. Skyline problem

Detail perhitungan kompleksitas algoritma yang memanfaatkan strategi Divide and Conquer dapat dilakukan secara rekursif,

$$T(n) = \begin{cases} g(n) & \text{if } n \leq n_0 \\ T(n_1) + T(n_2) + \ldots + T(n_r) + f(n) & \text{if } n > n_0 \end{cases}$$

- $T(n)$: kompleksitas waktu penyelesaian persoalan P yang berukuran n
- $g(n)$: kompleksitas waktu untuk menyelesaikan persoalan jika n sudah berukuran kecil
- $T(n_1) + T(n_2) + \ldots + T(n_r)$: kompleksitas waktu untuk memproses setiap upa-persoalan
- $f(n)$: kompleksitas waktu untuk menggabungkan solusi dari masing-masing upa-persoalan
- Tahap membagi persoalan menjadi upa-persoalan dapat dilakukan dalam O(1), sehingga tidak dimasukkan ke dalam formula

Teorema Master dapat digunakan untuk menentukan notasi asimptotik kompleksitas waktu yang berbentuk relasi rekurens dengan mudah tanpa harus menyelesaikannya secara iteratif [**?**]. Teorema tersebut memiliki pernyataan, misalkan $T(n)$ adalah fungsi monoton menaik yang memenuhi relasi rekurens:

$$T(n) = aT(\frac{n}{b}) + cn^d$$

Yang memenuhi,

- $n = bk$
- $k = 1, 2, 3, \ldots$
- $a \geq 1$
- $b \geq 2$
- $c, d \geq 0$

Maka nilai $T(n)$ dapat ditentukan menggunakan bentuk fungsi,

$$T(n) = \begin{cases} n^d & \text{if } a < b^d \\ n^d \log n & \text{if } a = b^d \\ n^{\log_b a} & \text{if } a > b^d \end{cases}$$

2.2 Teorema Sumbu Pemisah

Teorema Sumbu Pemisah (yang biasa juga disebut sebagai *separating axis theorem* oataupun *the Separating Hyperplane Theorem*) adalah teorema yang dikemukakan oleh Herman Minkowski, teori ini berkaitan dengan hubungan objek geometris dua dimensi yang bertipe cembung.

Sebuah bentuk geometris dua dimensi (*polygon*) dikatakan sebagai bertipe cembung jika untuk setiap garis yang dapat digambar pada bidang dua dimensi, maka garis tersebut tidak akan memotong (*intersect*) bentuk tersebut lebih dari dua kali. Kebalikan dari bentuk bertipe cembung yaitu bentuk bertipe cekung (*concave*).

Teorema sumbu pemisah mengemukakan bahwa sebuah bentuk pada dimensi ke-n tidak saling tumpang tindih jika terdapat sumbu yang dapat memisahkan keduanya, atau dalam kata lain dapat digambar sebuah objek dua dimensi (garis) sehingga tidak terdapat objek satu dimensi (titik) pada garis tersebut yang juga merupakan titik dari kedua objek pada dimensi ke-n tersebut. Umumnya kita hanya perlu memperhatikan kasus pada dimensi kedua dan ketiga.

Secara teoritis terdapat tak hingga sumbu yang dapat kita gunakan untuk mengecek ketumpang-tindihan antara dua poligon (objek dua dimensi) atau polihedra (objek tiga dimensi). Namun, kita hanya perlu mengecek sumbu yang tegak lurus dengan permukaan dari poligon atau polihedra, misalnya pada poligon kita hanya perlu mengecek apakah sumbu yang tegak lurus dengan salah satu sisi poligon dapat memisahkan kedua poligon, hal yang sama juga berlaku untuk polihedra (digunakan sumbu yang tegak lurus dengan permukaan polihedra). Sumbu yang tegak lurus dengan sisi poligon atau permukaan polihedra dapat dicari dengan operasi *cross product* vektor.

Untuk mengecek apakah sebuah sumbu dapat memisahkan dua poligon atau polihedra, setiap titik sudut pada poligon atau polihedra perlu diproyeksikan ke sumbu tersebut dengan menggunakan operasi *dot product* vektor, setelah dioperasikan maka permasalahan berubah menjadi apakah terdapat tumpang tindih antara dua garis

pada sumbu yang sama. Jika terdapat setidaknya satu hasil proyeksi yang tidak saling tumpang tindih, maka pastilah kedua bentuk saling terpisah. Untuk memudahkan perhitungan, salah satu bentuk dapat dinormalisasikan posisi-nya ke daerah *origin* (0, 0) atau (0, 0, 0).

2.3 Octree dan Voxelization

Voxels adalah satuan unit diskrit yang merepresentasikan sebuah elemen volume dalam lingkungan 3 dimensi, voxel memiliki analogi yang sama terhadap pixel yang merupakan satuan unit diskrit luas pada lingkungan 2 dimensi, sebuah voxel memiliki properti paling dasar berupa titik-titik koordinat titik sudut (x, y, z) dari bentuk satuan volume.

Polygon Mesh adalah kumpulan simpul (*vertice*), tepi *edge*, dan permukaan *face* yang menentukan bentuk permukaan objek polihedral. Dalam grafika komputer, mesh digambarkan dalam model bingkai kawat (*wireframe* dibentuk secara kontinu. Permukaan dari mesh biasanya terdiri dari segitiga, ataupun segi empat. Salah satu bentuk reprsentasi mesh yang dapat digunakan yaitu *face-vertex* mesh, dimana sebuah objek didefnisikan sebagai himpunan *vertice* dan himpunan *face*. Representasi tersebut banyak digunakan dalam format *file* grafika komputer 3D, misalnya format file *obj*.

Voxelization adalah proses mengubah struktur data yang menyimpan informasi geometris dalam domain kontinu (seperti mesh *face-vertex* yang terdiri dari segitiga) menjadi objek dalam bentuk himpunan *voxel* yang diskrit. Teknik ini sangat berguna karena komputasi grafika komputer dapat memanfaatkan properti diskrit untuk berbagai hal, seperti deteksi tabrakan, simulasi presisi, ataupun aestetik objek.

Proses *voxelization* dapat dilakkan secara efisien menggunakan struktur data Octree. Octree adalah salah satu turunan struktur data dalam bentuk *tree*, dimana setiap *node* pada *tree* tepat memiliki 8 anak. Properti Octree yang tepat membagi *node* menjadi 8 *node* lainnya dapat dimanfaatkan untuk membagi sebuah ruang 3 dimensi menjadi 8 ruang 3 dimensi lebih kecil yang saling berdekatan. Sehingga, dengan cara tersebut dapat dilakukan *voxelization* dengan strategi Divide and Conquer.

Pembentukan mesh yang ter-voxelisasi dapat dilakukan dengan melakukan algo-

ritma berikut,

1. Tentukan daerah volume awal (yang akan menjadi *root* dari octree), yaitu volume terkecil yang mencakup semua poligon pembentuk *mesh*
2. Cek apakah kedalaman *octree* sudah sesuai dengan preferensi
3. Cek apakah daerah volume saat ini berpotongan dengan setidaknya salah satu poligon dari *mesh*
4. Jika iya, bagi daerah volume saat ini menjadi 8 daerah volume lebih kecil yang merata, masukkan 8 daerah volume tersebut sebagai anak dari *node* saat ini, untuk setiap anak jadikan daerah volume saat ini dan kembali ke langkah (2)
5. Jika tidak, *node* saat ini tidak akan membentuk dibagi menjadi daerah volume lebih kecil.

2.4 Parallel Computing

Paradigma komputasi parallel (*parallel computing*) adalah teknik komputasi dengan cara menggunakan lebih dari satu sumber daya komputasi di saat yang bersamaan. Paradigma ini dapat digunakan untuk menyelesaikan masalah dengan cara memecah permasalahan menjadi upa-permasalahan lebih kecil yang independen satu sama lain dan hasilnya dapat digabungkan menjadi solusi global, kemudian upa-permasalahan dibagi kepada sumber daya komputasi berbeda, dan penemuan solusi dilakukan bersamaan dengan teknik sinkronisasi tertentu.

Kemampuan komputasi parallel dapat dihasilkan dengan beberapa cara misalnya,

- Lebih dari satu unit fungsional (cache L1, cache L2, cabang, prefetch, decode, floating-point, pemrosesan grafis (GPU), integer, dll.)
- Lebih dari satu unit/*core* eksekusi
- Lebih dari satu *core* perangkat keras
- Jaringan komputer yang terdiri dari banyak komputer dan berdiri sendiri untuk membuat cluster komputer parallel yang lebih besar.

Aplikasi komputasi parallel dalam suatu perangkat keras merupakan tanggung jawab sistem operasi perangkat keras tersebut. Umumnya sistem operasi akan

mengimplementasikan mekanisme *context switching* yang dapat memberikan manfaat *concurrency* dan *parallellism*. *Concurrency* yaitu kemampuan untuk mengerjakan banyak proses dalam waktu yang sama dengan cara berganti proses secara cepat, tanpa sumber daya komputasi tambahan. Sedangkan *Parallelism* adalah kemampuan untuk mengerjakan banyak proses dalam waktu yang sama dengan cara menggunakan sumber daya komputasi lainnya secara maksimal.

Aplikasi *parallel computing* yaitu *parallel programming*. Dalam pembuatan program komputer modern, unit terkecil eksekusi yang biasa digunakan yaitu *thread*, kumpulan *thread* yang memiliki ruang memori yang sama akan membentuk *process*. Penggunaan *multi-threading* dibandingkan *multi-processing* lebih diutamakan karena, *context switch* antar *thread* lebih murah secara komputasional. Selain itu *thread* juga saling berbagi bagian data *heap* dan *global* sehingga dapat lebih mudah untuk sinkronisasi.

Sinkronisasi perlu dilakukan antar unit eksekusi yang saling berbagi data pada alamat memori yang sama, jika tidak maka akan terjadi *race condition*. Teknik sinkronisasi yang dapat dilakukan antar unit eksekusi yaitu *spinlocking / busy waiting* dimana unit eksekusi yang sedang menunggu sumber daya akan menunggu sambil terus mengecek apakah sumber daya tersebut sudah tersedia. Kemudian terdapat *sleeping* dimana unit eksekusi akan meminta sistem operasi untuk mengehentikan unit eksekusi tersebut sampai sumber daya sudah tersedia.

2.5 3D Visualization

Dalam *graphics programming*, setiap titik di ruang dinyatakan dengan tiga komponen bilangan real yaitu x, y, dan z. Sistem koordinat yang digunakan umumnya mengikuti aturan *right-handed coordinate system*, di mana sumbu X menunjuk ke kanan, sumbu Y menunjuk ke atas, dan sumbu Z menunjuk ke arah penonton. Operasi-operasi dasar pada vektor dalam ruang 3D yang digunakan dalam program ini meliputi:

- Operasi penjumlahan dan pengurangan digunakan untuk menghitung posisi relatif antara dua titik dan jarak antar objek dalam *scene*.
- Perkalian skalar dimanfaatkan untuk menskalakan panjang sebuah vektor, misalnya dalam proses menghitung posisi kamera berdasarkan jarak orbit.
- Operasi *dot product* (perkalian titik) menghasilkan nilai skalar yang merepresentasikan proyeksi satu vektor terhadap vektor lain, dengan formula $\mathbf{a} \cdot \mathbf{b} =$

$a_x b_x + a_y b_y + a_z b_z$. Operasi ini diaplikasikan secara khusus pada tahap *back-face culling* dan *shading*.

- Operasi *cross product* (perkalian silang) menghasilkan vektor baru yang posisinya tegak lurus terhadap kedua vektor masukan. Kalkulasi ini sangat penting untuk membangun basis kamera (vektor *right* dan *up*) serta menentukan arah normal dari sebuah bidang.
- Proses normalisasi mengubah sebuah vektor menjadi vektor satuan (panjang 1) dengan cara membagi tiap komponennya dengan magnitudo vektor tersebut, yang sangat krusial untuk memastikan perhitungan *dot product* dan intensitas *shading* yang akurat.

Pada *visualizer* ini, ruang 3D direpresentasikan menggunakan struktur *Octree* yang membagi ruang menjadi sub-region yang berbentuk kubus. Setiap titik dalam ruang tersebut dinyatakan secara matematis sebagai vektor posisi $\mathbf{p} = (x, y, z)$. Terdapat dua ruang koordinat utama yang digunakan: *world space*, yang bersifat absolut sebagai ruang tempat seluruh objek ditempatkan, dan *camera space*, yang bersifat relatif terhadap posisi dan orientasi kamera, di mana sumbu Z diasumsikan selalu mengarah menjauhi kamera.

Visualizer memanipulasi *world space* yang bertindak sebagai ruang absolut tempat seluruh voxel ditempatkan, dan *camera space* yang bersifat relatif terhadap posisi dan orientasi kamera, di mana sumbu Z selalu mengarah menjauhi kamera dan diimplementasikan melalui fungsi `getCameraPosition()`. Dengan pendekatan ini, kamera dimungkinkan untuk terus mengorbit pada satu titik target dengan menggunakan koordinat bola (*spherical coordinates*). Posisi absolut kamera dikalkulasi berdasarkan tiga parameter utama, yakni rotasi horizontal (*yaw*, ψ), rotasi vertikal (*pitch*, ϕ), dan jarak linear kamera dari target (d), yang diformulasikan sebagai berikut:

$$\mathbf{p}_{\text{cam}} = \mathbf{p}_{\text{target}} + d \begin{pmatrix} \cos\phi \cdot \sin\psi \\ \sin\phi \\ \cos\phi \cdot \cos\psi \end{pmatrix}$$

Setelah posisi kamera berhasil didapatkan, pengguna dapat menggeser kursor untuk mengubah ψ dan ϕ, serta melakukan *scroll* untuk menyesuaikan d, sehingga pandangan secara dinamis diperbarui setiap saat. Selanjutnya, sistem perlu melakukan transformasi pandangan (*view transform*) guna memetakan koordinat dari *world space* ke *camera space*. Proses yang dieksekusi di dalam fungsi `transformToCamera()` ini dilakukan dengan membangun basis ortogonal kamera yang terdiri dari tiga vektor satuan, yaitu *forward* ($\mathbf{f}$), *right* ($\mathbf{r}$), dan *up* ($\mathbf{u}$). Ketiga vektor ini dihitung

melalui serangkaian *cross product* vektor:

$$\mathbf{f} = \frac{\mathbf{p}_{\text{target}} - \mathbf{p}_{\text{cam}}}{|\mathbf{p}_{\text{target}} - \mathbf{p}_{\text{cam}}|}$$

$$\mathbf{r} = \frac{\mathbf{f} \times \mathbf{up}_{\text{world}}}{|\mathbf{f} \times \mathbf{up}_{\text{world}}|}$$

$$\mathbf{u} = \mathbf{r} \times \mathbf{f}$$

Koordinat sebuah titik $\mathbf{p}$ dalam *camera space* kemudian diperoleh dengan memproyeksikan vektor relatif $\mathbf{v} = \mathbf{p} - \mathbf{p}_{\text{cam}}$ ke masing-masing vektor basis tersebut, sehingga menghasilkan:

$$\mathbf{p}_c = \begin{pmatrix} \mathbf{v} \cdot \mathbf{r} \\ \mathbf{v} \cdot \mathbf{u} \\ \mathbf{v} \cdot \mathbf{f} \end{pmatrix} = \begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix}$$

Keseluruhan dari proses visualisasi dikelola oleh *rendering pipeline* di dalam fungsi `render()`. Fungsi `project3Dto2D()` diimplementasikan untuk mensimulasikan perspektif, di mana objek yang berada lebih jauh dari kamera akan tampak lebih kecil. Titik dalam *camera space* dengan koordinat (x_c, y_c, z_c) diproyeksikan ke kanvas 2D menggunakan formula berikut:

$$x_{\text{screen}} = \left(\frac{f_{\text{fov}} \cdot x_c}{z_c \cdot a} \right) \cdot \frac{W}{2} + \frac{W}{2}$$

$$y_{\text{screen}} = \left(\frac{-f_{\text{fov}} \cdot y_c}{z_c} \right) \cdot \frac{H}{2} + \frac{H}{2}$$

Pada kalkulasi proyeksi tersebut, $f_{\text{fov}} = 1/\tan(\theta/2)$ merupakan faktor *Field of View* dengan sudut pandang $\theta = 60°$, $a = W/H$ adalah rasio aspek layar, sementara W dan H masing-masing adalah lebar dan tinggi jendela aplikasi dalam piksel. Tanda negatif pada y_{screen} diperlukan karena sumbu Y piksel bertumbuh ke bawah, sedangkan sumbu Y dunia bertumbuh ke atas. Efek perspektif secara matematis tercipta langsung dari pembagian koordinat dengan nilai kedalaman z_c, dan titik dengan $z_c < 0{,}1$ dibuang untuk menghindari pembagian dengan nol (*near-plane clipping*).

Teknik *back-face culling* digunakan untuk mengabaikan *face* voksel yang membelakangi kamera sekaligus mengoptimalkan komputasi yang dilakukan. Arah pandang dari kamera menuju *face* didefinisikan sebagai:

$$\hat{\mathbf{d}} = \frac{\mathbf{p}_{\text{face}} - \mathbf{p}_{\text{cam}}}{|\mathbf{p}_{\text{face}} - \mathbf{p}_{\text{cam}}|}$$

Evaluasi dilakukan dengan memeriksa hasil kali titik (*dot product*) antara vektor normal permukaan ($\hat{\mathbf{n}}$) dan vektor arah pandang ($\hat{\mathbf{d}}$) tersebut. Kondisi *back-face* dipenuhi apabila:

$$\hat{\mathbf{n}} \cdot \hat{\mathbf{d}} > 0$$

Jika hasil evaluasi lebih besar dari nol, *face* tersebut dibuang karena dipastikan tidak terlihat oleh pengguna. Teknik ini secara efektif memangkas beban kalkulasi poligon hingga separuhnya. Untuk *face-face* yang berhasil lolos tahap *culling*, diselesaikan menggunakan *Painter's Algorithm*. Algoritma ini bekerja dengan menghitung nilai kedalaman z_c tiap *face*, lalu mengurutkannya dari posisi terjauh hingga terdekat menggunakan metode pengurutan dengan kompleksitas $O(n \log n)$. *Face-face* tersebut kemudian digambar secara berurutan, sehingga objek yang berada di depan secara alamiah akan menimpa piksel objek yang ada di belakangnya.

Kecerahan warna tiap poligon dihitung menggunakan model pencahayaan *Lambertian* (*diffuse shading*). Intensitas cahaya yang diterima suatu permukaan berbanding lurus dengan kosinus sudut antara normal permukaan dan arah cahaya, yang dikalkulasi dengan:

$$I = \max\big(0,\ \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}\big)$$

Di mana $\hat{\mathbf{l}}$ merupakan vektor arah sumber cahaya yang telah dinormalisasi. Nilai intensitas I kemudian digunakan untuk menghitung kecerahan akhir (B) dengan menggabungkan komponen *ambient* dasar (A):

$$B = A + (1 - A) \cdot I$$

Dengan nilai $A = 0{,}25$, bagian yang tidak terkena cahaya secara langsung tetap mendapatkan kecerahan sebesar 25%, sehingga detail geometri tetap terlihat dari semua sudut pandang.

3 Impelementasi

Implementasi dilakukan dengan bahasa pemrograman C++ yang berorientai objek, *Multimedia*/GUI *framework* SFML versio 3.0.2 dan dengan menggunakan *build system* Cmake serta *build tools* Make.

3.1 Voxelization

Voxelisasi dengan menggunakan Octree dilakukan dengan membuat Node Octree yang akan menyimpan,

1. tipe node tersebut (node internal, node leaf kosong, node leaf berisi volume mesh)
2. volume representasi node tersebut (dalam bentuk *Axis-Aligned Bounding Box* yang terdiri dari dua titik sudut minimal dan maksimal dari Bounding Box)
3. 8 pointer ke anak dari node

Alur program secara garis besar yaitu sebagai berikut,

1. Ambil input dari pengguna menggunakan CLI dan lakukan validasi, jika semua pilihan pengguna valid maka pergi ke langkah selanjutnya
2. Lakukan pembacaan dan *parsing* dari file masukan, jika terdapat *error* maka berhentikan program dengan pesan di CLI
3. Lakukan vokselisasi dengan memetakan *vertex-face mesh* ke octree
4. Gunakan octree untuk membuat file keluaran, hilangkan *face* ataupun *vertex* duplikat jika dibutuhkan
5. Tampilkan file keluaran pada 3D viewer jika sesuai pilihan pengguna

Langkah-langkah algoritma voxelisasi yang digunakan yaitu,

1. Buat node root dari tree, tentukan volume dari node tersebut dengan melihat *minima* dan *maxima* dari setiap poligon segitiga mesh
2. Mulai lakukan rekursi secara iteratif menggunakan struktur data *stack* yang berisi pointer ke node saat ini dan kedalaman saat ini
3. Tentukan daerah volume dari setiap anak node saat ini,
4. Untuk setiap anak, cek apakah daerah volume setiap ini berpotongan dengan setidaknya salah satu poligon dari *mesh*

5. Jika iya, cek apakah kedalaman saat ini sudah paling dalam, jika belum maka tandai node anak tersebut sebagai node internal, jika sudah maka tandai sebagai node yang berisi volume dan masukkan pointer ke node anak tersebut ke stack beserta kedalamannya, bagian ini merupakan bagian *divide* dari algoritma Divide and Conquer, kemudian kembali ke langkah (3)

6. Jika tidak, maka tandai bagian ini sebagai node internal

Hal yang perlu diperhatikan dari proses voxelisasi ini yaitu tidak ada proses *combine* dari algoritma Divide and Conquer, tujuan utama yaitu hanya konstruksi daerah volume pada kedalaman tree paling dalam, tidak ada proses penggabungan volume dari node terdalam ataupun proses sejenisnya. Proses *combine* kemungkinan dapat diimplementasikan untuk algoritma grafika komputer yang lebih canggih terkait voxelisasi.

Pengecekan perpotongan antara Bounding Box dan poligon segitiga adalah masalah pencarian perpotongan antara poligon dan polihedra. Akan digunakan teorema sumbu pemisah dengan dilakukan pengecekan 13 sumbu berikut ini [**?**],

- sumbu yang melewati garis normal poligon segitiga
- sumbu-X, sumbu-Y, dan sumbu-Z,
- sumbu yang tegak lurus dengan setiap kombinasi pasangan (sumbu-X, sumbu-Y, sumbu-Z) dan ketiga sisi segitiga, jumlah sumbu tersebut yaitu $3 \cdot 3 = 9$

Jika terdapat satu sumbu yang dapat memisahkan poligon segitiga dan polihedra balok, maka akan dianggap sebagagi tidak berpotongan, begitu pula sebaliknya.

Terdapat implementasi penghilangan data *vertex* ataupun *face* duplikat dengan memanfaatkan struktur data *Red-Black Tree* implementasi STL C++ dalam bentuk *std::set* dan *std::map*.

3.2 Concurrency

Program yang dibuat secara *default* akan menggunakan *concurrency*, jika pengguna tidak ingin memakai *concurrency* maka harus memilih satu *thread* sebagai pilihan banyaknya *thread*.

Concurrency dilakukan dengan menggunakan sebuah *class* yang memegang data global bernama OctreeContext. Kelas tersebut akan memiliki sebuah *stack* dalam yang setiap entrinya dapat diisi OctreeNode dan informasi lainnya yang didseain

dengan fitur *templating* dari C++. Didefinisikan sebuah *mutex* umum dan 4 *mutex* spesifik pada ranah global, dan satu mutex khusus yang dipakai Octree.

Setiap operasi konkurren akan menggunakan *std::thread*. Agar memudahkan implementasi dibuat sebuah metode *genericWorkerThread* pada kelas OctreeContext yang dapat dimodifikasi dengan fleksibel. Kemudian mekanisme sinkronisasi menggunakan *std::condition_variable* untuk *sleeping* dan *std::unique_lock* bersamaan dengan *std::mutex* untuk mekanisme *spinlocking/busy waiting*.

4 Hasil Eksperimen

article graphicx float

Output apple.obj (Depth = 8)

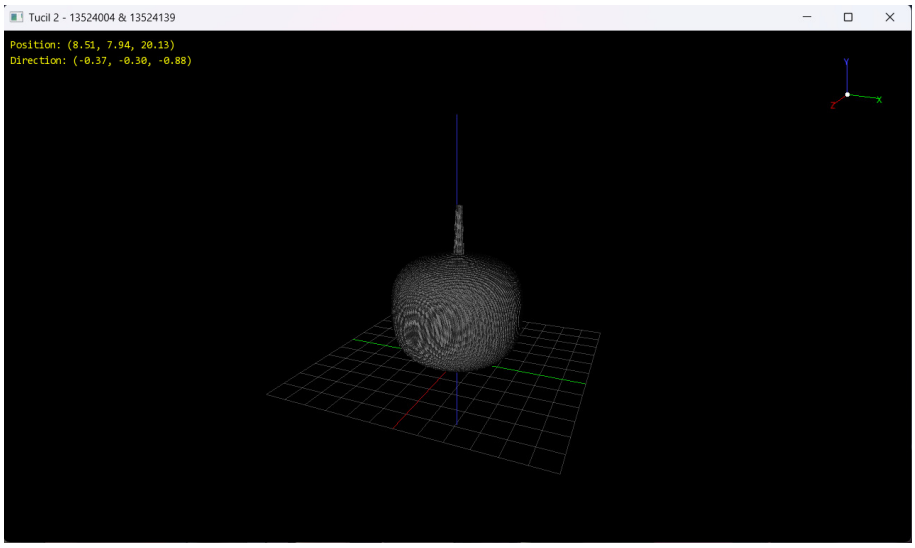


Figure 1: Output apple.obj (Depth = 8)

Input apple.obj (Depth = 8)

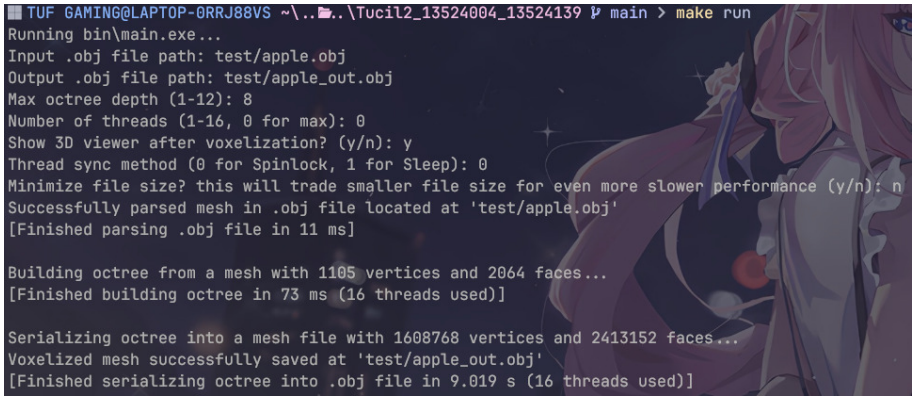


Figure 2: Input apple.obj (Depth = 8) – in1

Figure 3: Input apple.obj (Depth = 8) – in2

Output GUI cow.obj (Depth = 8)

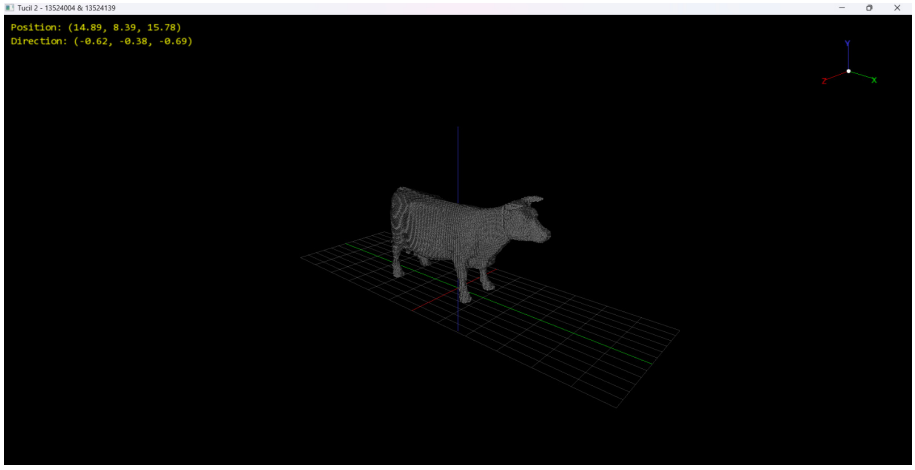


Figure 4: Output GUI cow.obj (Depth = 8)

Input cow.obj (Depth = 8)

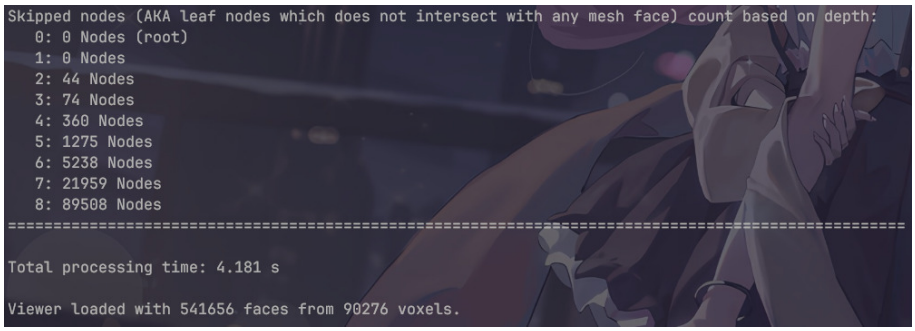


Figure 5: Input cow.obj (Depth = 8) – in2

Output line.obj (Depth = 8)

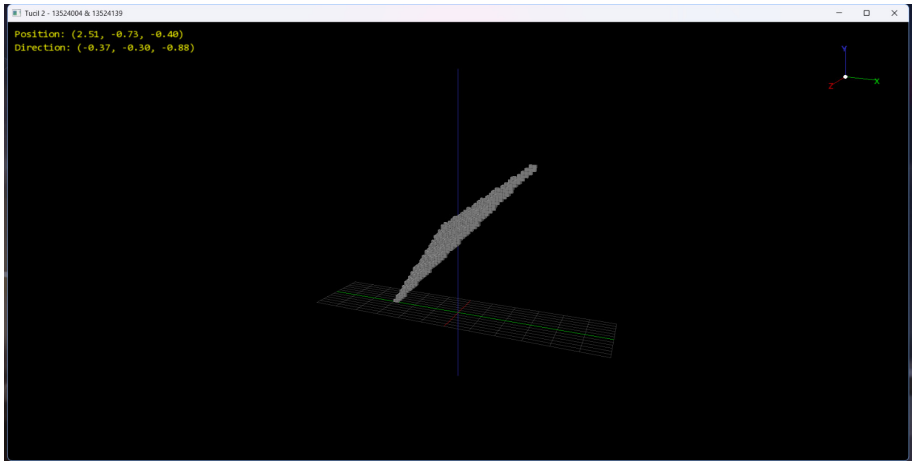


Figure 6: Output line.obj (Depth = 8)

Input line.obj (Depth = 8)

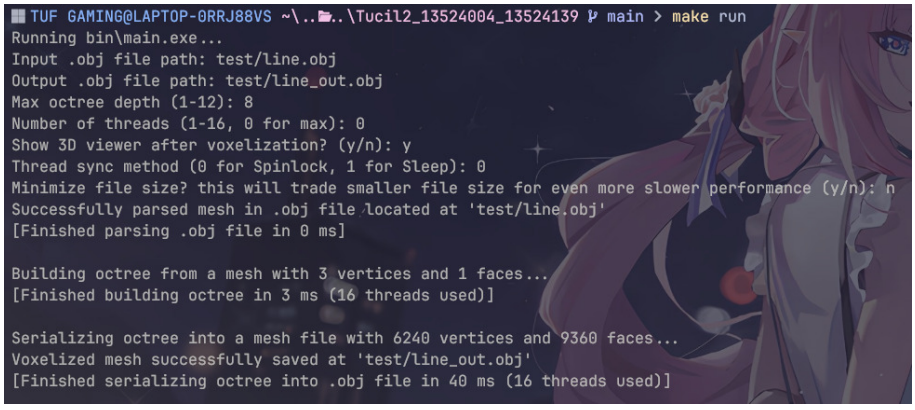


Figure 7: Input line.obj (Depth = 8) – in1

Figure 8: Input line.obj (Depth = 8) – in2

Output pumpkin.obj (Depth = 8)

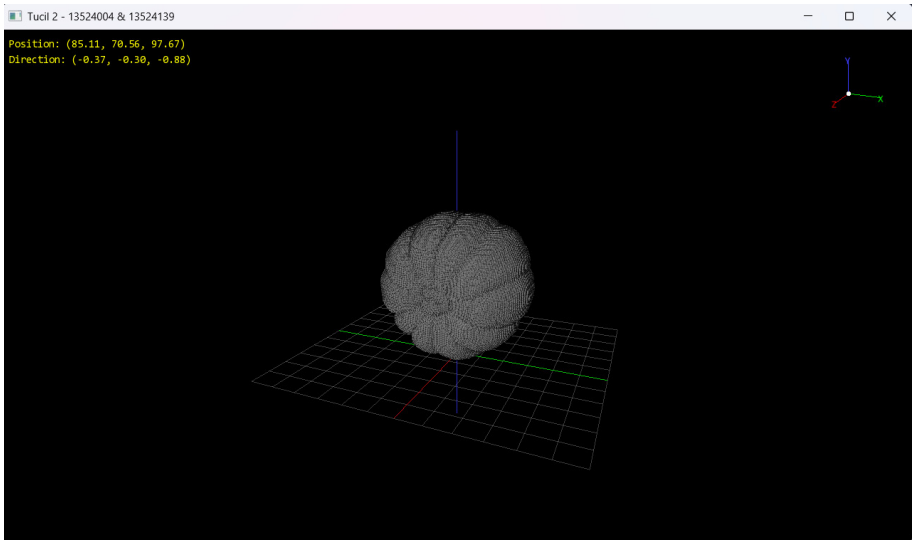
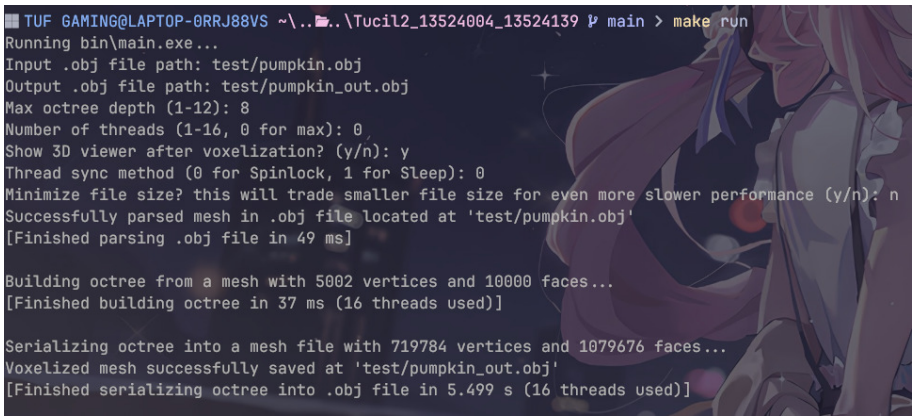


Figure 9: Output pumpkin.obj (Depth = 8)

Input pumpkin.obj (Depth = 8)



```
TUF GAMING@LAPTOP-0RRJ88VS ~\..\..\Tucil2_13524004_13524139 main > make run
Running bin\main.exe...
Input .obj file path: test/pumpkin.obj
Output .obj file path: test/pumpkin_out.obj
Max octree depth (1-12): 8
Number of threads (1-16, 0 for max): 0
Show 3D viewer after voxelization? (y/n): y
Thread sync method (0 for Spinlock, 1 for Sleep): 0
Minimize file size? this will trade smaller file size for even more slower performance (y/n): n
Successfully parsed mesh in .obj file located at 'test/pumpkin.obj'
[Finished parsing .obj file in 49 ms]

Building octree from a mesh with 5002 vertices and 10000 faces...
[Finished building octree in 37 ms (16 threads used)]

Serializing octree into a mesh file with 719784 vertices and 1079676 faces...
Voxelized mesh successfully saved at 'test/pumpkin_out.obj'
[Finished serializing octree into .obj file in 5.499 s (16 threads used)]
```

Figure 10: Input pumpkin.obj (Depth = 8) – in1

```
========================================= OCTREE STATISTICS =========================================
Octree max depth: 8
Number of voxel formed: 89973
Total number of vertices: 719784
Total number of faces: 1079676
Nodes count based on depth:
   0: 1 Nodes (root)
   1: 8 Nodes
   2: 32 Nodes
   3: 192 Nodes
   4: 608 Nodes
   5: 2768 Nodes
   6: 10864 Nodes
   7: 44656 Nodes
   8: 179736 Nodes
Skipped nodes (AKA leaf nodes which does not intersect with any mesh face) count based on depth:
   0: 0 Nodes (root)
   1: 4 Nodes
   2: 8 Nodes
   3: 116 Nodes
   4: 262 Nodes
   5: 1410 Nodes
   6: 5282 Nodes
   7: 22189 Nodes
   8: 89763 Nodes
=====================================================================================================

Total processing time: 5.553 s

Viewer loaded with 539838 faces from 89973 voxels.
```

Figure 11: Input pumpkin.obj (Depth = 8) – in2

Output teapot.obj (Depth = 8)

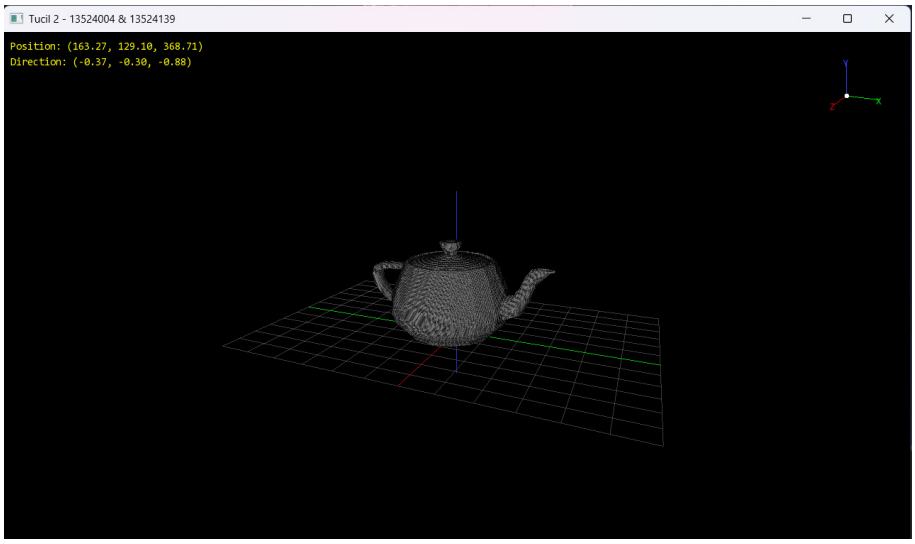


Figure 12: Output teapot.obj (Depth = 8)

Input teapot.obj (Depth = 8)

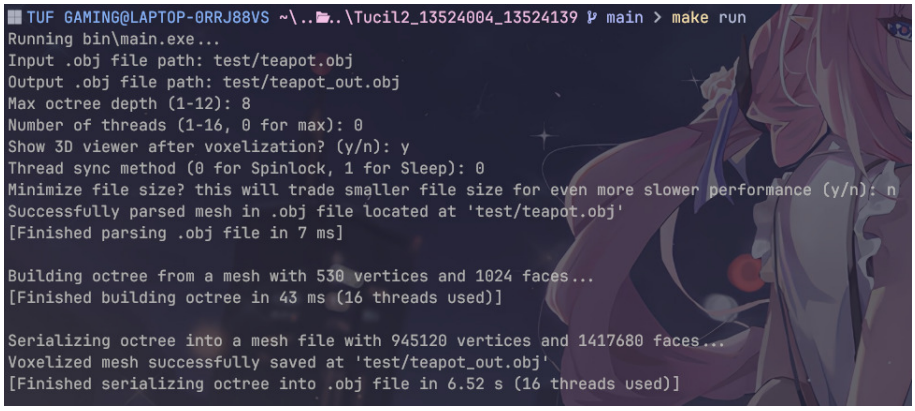


Figure 13: Input teapot.obj (Depth = 8) – in1

Figure 14: Input teapot.obj (Depth = 8) – in2

Output teddy.obj (Depth = 8)

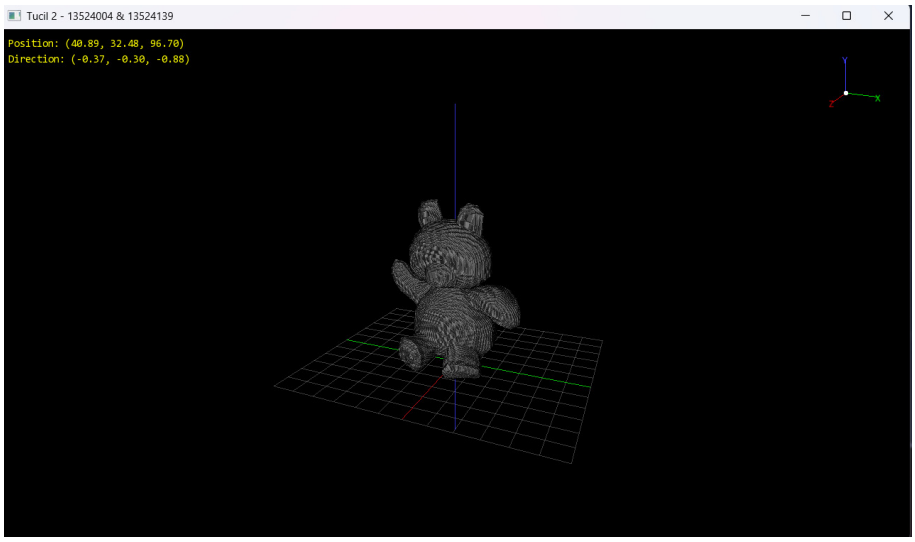


Figure 15: Output teddy.obj (Depth = 8)

Input teddy.obj (Depth = 8)

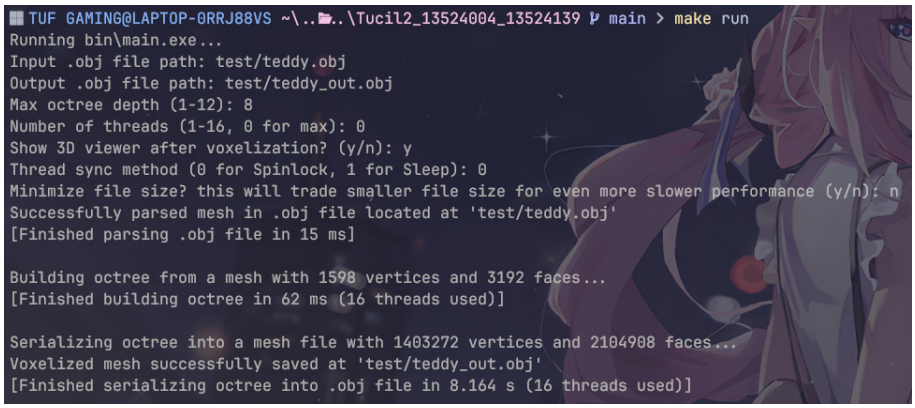


Figure 16: Input teddy.obj (Depth = 8) – in1

Figure 17: Input teddy.obj (Depth = 8) – in2